المملكة المغربية
ⵜⴰⴳⵍⴷⵉⵜ ⵏ ⵍⵎⵖⵔⵉⴱ
وزارة التربية الوطنية
والتعليم الأولي والرياضة
ⵜⴰⵎⴰⵡⴰⵙⵜ ⵏ ⵓⵙⴳⵎⵉ ⴰⵏⴰⵎⵓⵔ
ⴷ ⴰⵙⵙⵍⵎⴷ ⴰⵎⵣⵡⴰⵔⵓ ⴷ ⵜⵓⵏⵏⵓⵏⵜ

Examen National du Brevet de Technicien Supérieur (BTS)

Session 2026

- Sujet -

Filière :	**Développement des Applications Informatiques -DAI-**	**Durée :** 2 heures
Épreuve :	**Développement Mobile Android**	**Coefficient :** 45

CONSIGNES

— Le sujet comporte 4 dossiers.
— Il sera tenu compte de la qualité de rédaction.
— Aucun document n'est autorisé.

BARÈME DE NOTATION

Dossier	Intitulé	Points
1	Structure & Interface Utilisateur	10
2	Cycle de vie & Stockage local	10
3	Listes & Base de données	10
4	Firebase & Application complète	10
Total		**40**

CONTEXTE

On souhaite développer une application Android appelée :
SmartEvent Manager
Cette application permet de :
— Gérer des événements (réunions, conférences)
— Marquer un événement comme important
— Afficher les événements dans un StackView
— Sauvegarder les données localement et en ligne

CONSIGNES

— 4 dossiers
— Justifier les réponses
— Code clair et structuré exigé

DOSSIER I : UI & ÉVÉNEMENTS (10 pts)

1. Interface (3 pts)

Créer une interface contenant :

— EditText (titre événement)
— CheckBox (événement important)
— Bouton Ajouter
— StackView pour afficher les événements

Travail demandé :

1. Donner le XML complet en utilisant une constraint layout
2. Expliquer le rôle du StackView

DOSSIER II : STOCKAGE & SERIALIZABLE (10 pts)

1. Modèle (3 pts)

Créer une classe Event :
— id
— titre
— important (boolean)
— date (long)

2. Fichiers (4 pts)

1. Sauvegarder une liste d'événements
2. Lire la liste depuis fichier

3. SharedPreferences avancé (3 pts)

On souhaite gérer :
— Nombre total d'événements (int)
— Mode notifications (boolean)
— Suppression
— Suppression totale

DOSSIER III : ADAPTER PERSONNALISÉ & SQLITE (10 pts)

1. SQLite (4 pts)

1. Créer DBHelper
2. Créer table Event
3. Implémenter :
   — ajout d'un nouvel événement
   — modification d'un événement
   — suppression d'un événement
   — récupération de tous les évenements
   — récupération de tous les évenements importants

2. Adapter personnalisé (4 pts)

1. Créer un Adapter personnalisé pour StackView qui récupère les données à partir de la base de données
2. Expliquer le principe du **ViewHolder**

Question théorique : Pourquoi un adapter personnalisé est-il plus performant ?

3. Gestion des événements UI (4 pts)

1. Gérer le clic du bouton Ajouter (appel vers SQLite)
2. Gérer l'état du CheckBox
3. Gérer le clic sur un élément du StackView (affichage Toast)
4. Gérer le long click (appel suppression via SQLite)

4. Gestion des exceptions (2 pts)

Gérer obligatoirement :
— NumberFormatException (saisie utilisateur)
— Implémenter un TextWatcher sur l'EditText, Afficher une erreur si le titre de l'événement dépasse 20 caractères

DOSSIER IV : FIREBASE & MVVM (10 pts)

1. Firebase (4 pts)

1. Ajouter un événement avec push()
2. Ajouter un noeud avec child("events")
3. Mettre à jour
4. Supprimer
5. Lire les données

2. Architecture MVVM (3 pts)

Expliquer :
— DAO
— Repository
— ViewModel

3. Code MVVM (3 pts)

DAO :

```
public class EventDao {

    private SQLiteDatabase db;

    public EventDao(SQLiteDatabase db){
        this.db = db;
    }

    public long insert(Event e){
        ContentValues values = new ContentValues();
        values.put("titre", e.getTitre());
        values.put("important", e.isImportant());
        return db.insert("event", null, values);
    }
```

```
    public Cursor getAll(){
        return db.rawQuery("SELECT * FROM event", null);
    }

    public int update(Event e){
        ContentValues values = new ContentValues();
        values.put("titre", e.getTitre());
        return db.update("event", values, "id=?", new String[]{String.valueOf(e.getId())})
    }

    public int delete(int id){
        return db.delete("event", "id=?", new String[]{String.valueOf(id)});
    }
}
```

Repository :

```
public class EventRepository {

    private EventDao dao;

    public EventRepository(EventDao dao){
        this.dao = dao;
    }

    public void insert(Event e){
        dao.insert(e);
    }
}
```

ViewModel :

```
public class EventViewModel extends ViewModel {

    private EventRepository repo;

    public EventViewModel(EventRepository repo){
        this.repo = repo;
    }

    public void addEvent(Event e){
        repo.insert(e);
    }
}
```

Activity :

```
viewModel.addEvent(new Event("Meeting", true));
```

Bon courage